

Mathematical Problem Solving I

Logic, Invariants, Colouring and Induction



Overview

1 Competitive Programming

2 Logic

3 Invariants

4 Colouring Arguments

5 Induction and Monovariants



Competitive Programming

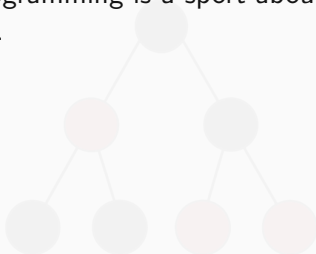


What Is Competitive Programming?



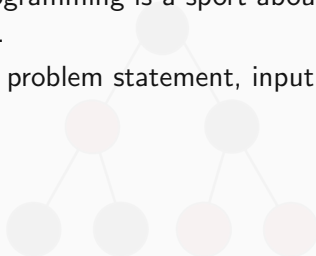
What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.



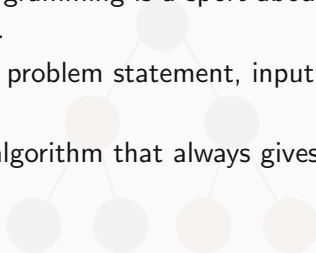
What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.
- You are given a problem statement, input, output, and constraints.

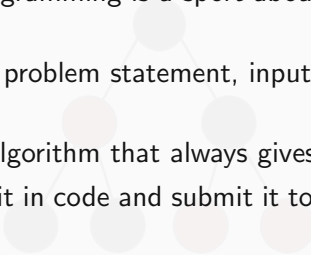


What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.
- You are given a problem statement, input, output, and constraints.
- You design an algorithm that always gives the right answer.



What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.
 - You are given a problem statement, input, output, and constraints.
 - You design an algorithm that always gives the right answer.
 - You implement it in code and submit it to an automatic judge.
- 

What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.
- You are given a problem statement, input, output, and constraints.
- You design an algorithm that always gives the right answer.
- You implement it in code and submit it to an automatic judge.
- The judge checks your program on many hidden test cases.

What Is Competitive Programming?

- Competitive programming is a sport about solving problems with algorithms.
- You are given a problem statement, input, output, and constraints.
- You design an algorithm that always gives the right answer.
- You implement it in code and submit it to an automatic judge.
- The judge checks your program on many hidden test cases.
- The challenge is not just solving it once. It is solving every valid case quickly enough.

Why Start With Mathematical Puzzles?



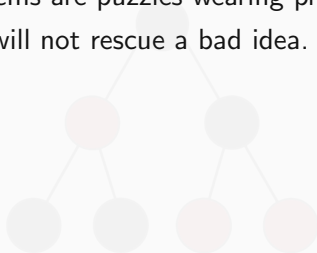
Why Start With Mathematical Puzzles?

→ Many CP problems are puzzles wearing programming clothes.



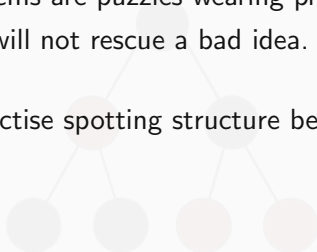
Why Start With Mathematical Puzzles?

- Many CP problems are puzzles wearing programming clothes.
- The computer will not rescue a bad idea. It will just run it faster.



Why Start With Mathematical Puzzles?

- Many CP problems are puzzles wearing programming clothes.
- The computer will not rescue a bad idea. It will just run it faster.
- We want to practise spotting structure before worrying about syntax.



Why Start With Mathematical Puzzles?

- Many CP problems are puzzles wearing programming clothes.
- The computer will not rescue a bad idea. It will just run it faster.
- We want to practise spotting structure before worrying about syntax.
- Today is about the thinking tools behind many algorithmic solutions.

Why Start With Mathematical Puzzles?

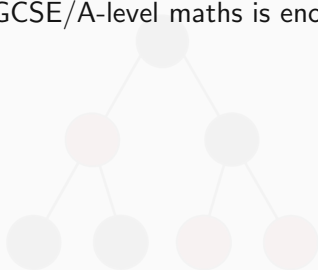
- Many CP problems are puzzles wearing programming clothes.
- The computer will not rescue a bad idea. It will just run it faster.
- We want to practise spotting structure before worrying about syntax.
- Today is about the thinking tools behind many algorithmic solutions.
- Once the idea is clear, implementation becomes much less mysterious.

What Do You Need To Start?



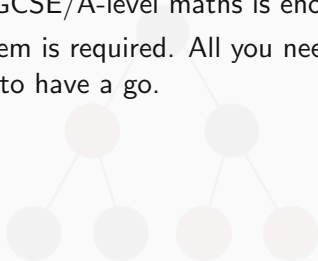
What Do You Need To Start?

→ Do not worry: GCSE/A-level maths is enough to get started.



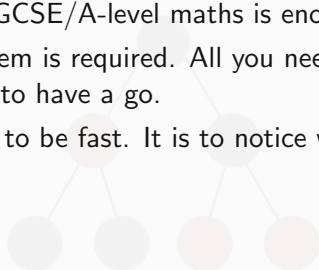
What Do You Need To Start?

- Do not worry: GCSE/A-level maths is enough to get started.
- No secret theorem is required. All you need is some patience and willingness to have a go.



What Do You Need To Start?

- Do not worry: GCSE/A-level maths is enough to get started.
- No secret theorem is required. All you need is some patience and willingness to have a go.
- The goal is not to be fast. It is to notice what is really going on.

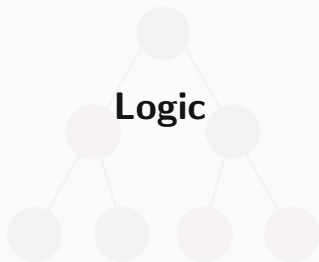


What Do You Need To Start?

- Do not worry: GCSE/A-level maths is enough to get started.
- No secret theorem is required. All you need is some patience and willingness to have a go.
- The goal is not to be fast. It is to notice what is really going on.
- A good solution should feel less like a trick, and more like finally seeing the shape of the problem.

What Do You Need To Start?

- Do not worry: GCSE/A-level maths is enough to get started.
- No secret theorem is required. All you need is some patience and willingness to have a go.
- The goal is not to be fast. It is to notice what is really going on.
- A good solution should feel less like a trick, and more like finally seeing the shape of the problem.
- If you already have a stronger foundation, we have left some harder questions for you to tackle.

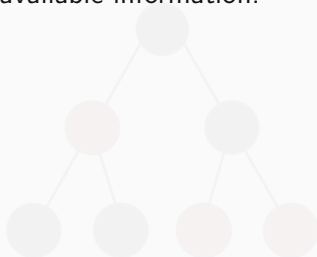


What is Logic?



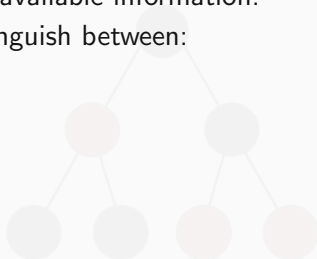
What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.



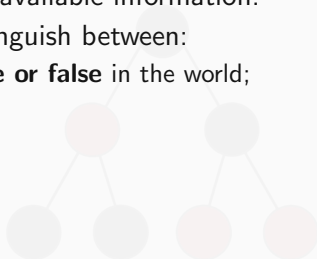
What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:



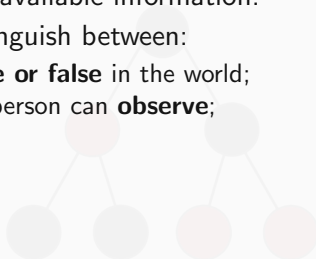
What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;



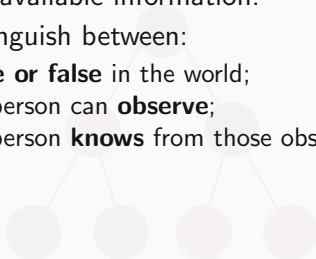
What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;



What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;
 - what each person **knows** from those observations;



What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;
 - what each person **knows** from those observations;
 - what each person knows about **what others know**;

What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;
 - what each person **knows** from those observations;
 - what each person knows about **what others know**;
 - what people can **infer** from each other's actions, statements, or information;

What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;
 - what each person **knows** from those observations;
 - what each person knows about **what others know**;
 - what people can **infer** from each other's actions, statements, or information;
 - what **silence, inaction, or missing information** can reveal.

What is Logic?

- Logic is the study of **reasoning**: how conclusions follow from assumptions or available information.
- It helps us distinguish between:
 - what is **true or false** in the world;
 - what each person can **observe**;
 - what each person **knows** from those observations;
 - what each person knows about **what others know**;
 - what people can **infer** from each other's actions, statements, or information;
 - what **silence, inaction, or missing information** can reveal.
- A central question is:

"Given what I know, and what I know about what others know, what must be true?"

Worked Example: Two Doors

You face two doors. One leads out; one does not.

One guard always tells the truth. The other always lies. You do not know which guard is which.

You may ask one guard one yes/no question. How do you find the safe door?

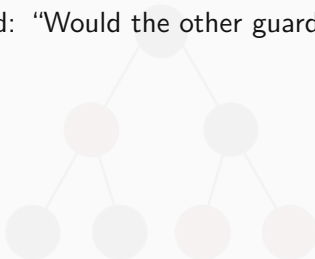
Logic

Solution: Two Doors



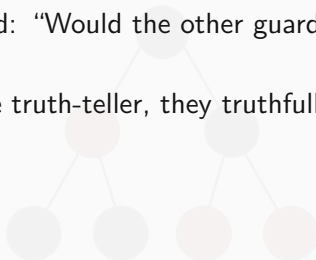
Solution: Two Doors

→ Ask either guard: “Would the other guard say this door is safe?”

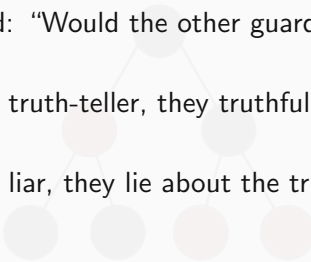


Solution: Two Doors

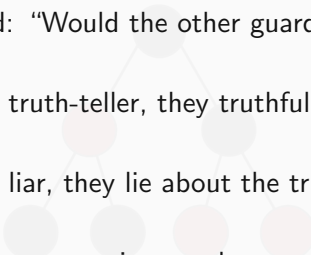
- Ask either guard: “Would the other guard say this door is safe?”
- If you asked the truth-teller, they truthfully report the liar’s false answer.



Solution: Two Doors

- 
- Ask either guard: “Would the other guard say this door is safe?”
 - If you asked the truth-teller, they truthfully report the liar’s false answer.
 - If you asked the liar, they lie about the truth-teller’s correct answer.

Solution: Two Doors

- 
- Ask either guard: “Would the other guard say this door is safe?”
 - If you asked the truth-teller, they truthfully report the liar’s false answer.
 - If you asked the liar, they lie about the truth-teller’s correct answer.
 - Either way, the answer points to the unsafe door. Choose the other one.

Student Exercises

- Basic: Same Birthday Month.
- Easy: Glove Selection.
- Medium: Two Doors.
- Hard: Islander Statements.
- Very Hard: Counterfeit Coin.

Invariants



What is an Invariants?

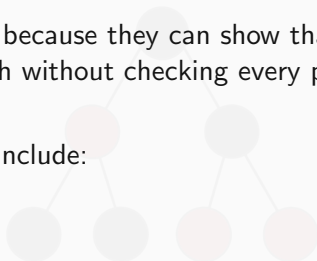


What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:



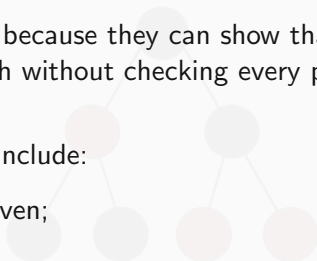
What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:

→ parity: odd or even;



What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:

- parity: odd or even;
- divisibility or remainder modulo m ;

What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:

- parity: odd or even;
- divisibility or remainder modulo m ;
- number of connected components;

What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:

- parity: odd or even;
- divisibility or remainder modulo m ;
- number of connected components;
- colour balance;

What is an Invariants?

An invariant is a quantity or property that stays unchanged after every legal move.

Invariants are useful because they can show that a certain position is impossible to reach without checking every possible sequence of moves.

Common invariants include:

- parity: odd or even;
- divisibility or remainder modulo m ;
- number of connected components;
- colour balance;
- XOR or binary properties.

Worked Example: Last Ball

A bag contains black and white balls.

Repeatedly remove two balls:

- same colours: put back one black ball;
- different colours: put back one white ball.

Can we predict the colour of the last ball?

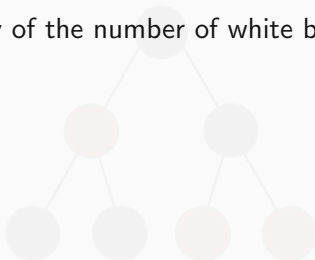
Invariants

Solution: Last Ball



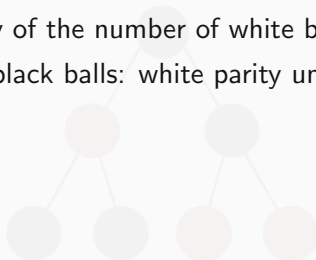
Solution: Last Ball

→ Track the parity of the number of white balls.



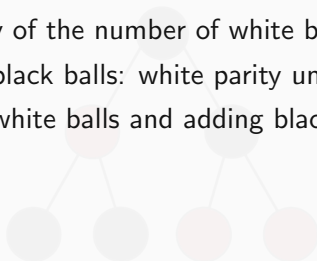
Solution: Last Ball

- Track the parity of the number of white balls.
- Removing two black balls: white parity unchanged.



Solution: Last Ball

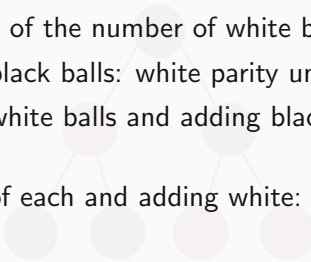
- Track the parity of the number of white balls.
- Removing two black balls: white parity unchanged.
- Removing two white balls and adding black: white count changes by -2 .



Solution: Last Ball

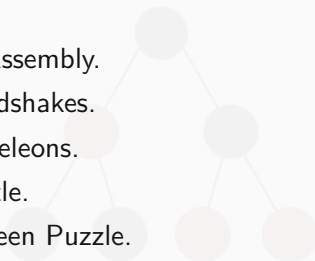
- Track the parity of the number of white balls.
- Removing two black balls: white parity unchanged.
- Removing two white balls and adding black: white count changes by -2 .
- Removing one of each and adding white: white count unchanged.

Solution: Last Ball

- 
- Track the parity of the number of white balls.
 - Removing two black balls: white parity unchanged.
 - Removing two white balls and adding black: white count changes by -2 .
 - Removing one of each and adding white: white count unchanged.
 - The parity of the number of white balls is invariant.

Student Exercises

- Basic: Jigsaw Assembly.
- Easy: Odd Handshakes.
- Medium: Chameleons.
- Hard: MU Puzzle.
- Very Hard: Fifteen Puzzle.



Colouring Arguments



What are Colouring Arguments?



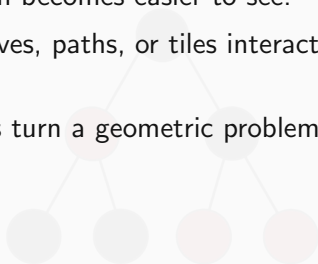
What are Colouring Arguments?

A **colouring argument** assigns colours to parts of a problem so that a hidden pattern becomes easier to see.

It is useful when moves, paths, or tiles interact with the colours in a predictable way.

Colouring arguments turn a geometric problem into a counting or parity problem.

Examples:



What are Colouring Arguments?

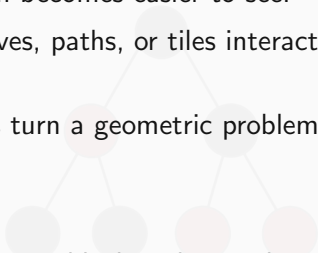
A **colouring argument** assigns colours to parts of a problem so that a hidden pattern becomes easier to see.

It is useful when moves, paths, or tiles interact with the colours in a predictable way.

Colouring arguments turn a geometric problem into a counting or parity problem.

Examples:

→ a domino covers one black and one white square;



What are Colouring Arguments?

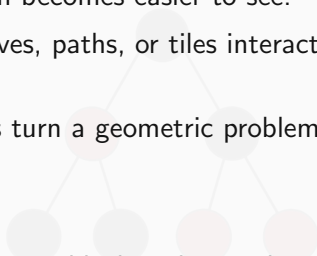
A **colouring argument** assigns colours to parts of a problem so that a hidden pattern becomes easier to see.

It is useful when moves, paths, or tiles interact with the colours in a predictable way.

Colouring arguments turn a geometric problem into a counting or parity problem.

Examples:

- a domino covers one black and one white square;
- a knight always moves to a square of the opposite colour;



What are Colouring Arguments?

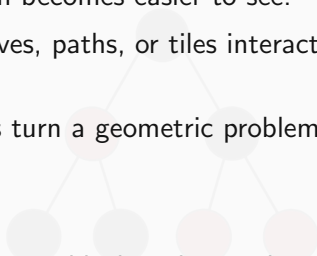
A **colouring argument** assigns colours to parts of a problem so that a hidden pattern becomes easier to see.

It is useful when moves, paths, or tiles interact with the colours in a predictable way.

Colouring arguments turn a geometric problem into a counting or parity problem.

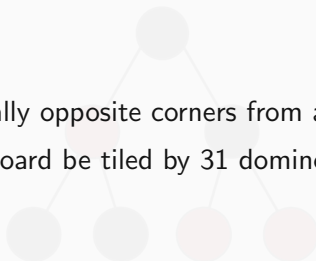
Examples:

- a domino covers one black and one white square;
- a knight always moves to a square of the opposite colour;
- a path through adjacent grid squares alternates between colours.



Worked Example: Deficient Chessboard

Remove two diagonally opposite corners from a chessboard.
Can the remaining board be tiled by 31 dominoes?

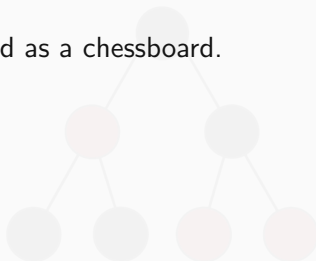


Solution: Deficient Chessboard



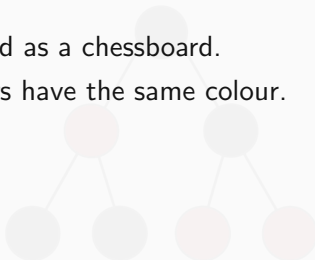
Solution: Deficient Chessboard

→ Colour the board as a chessboard.



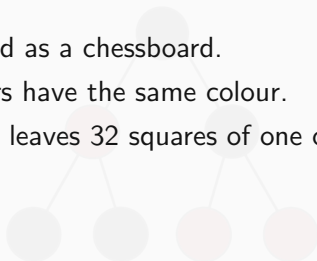
Solution: Deficient Chessboard

- Colour the board as a chessboard.
- Opposite corners have the same colour.



Solution: Deficient Chessboard

- Colour the board as a chessboard.
- Opposite corners have the same colour.
- Removing them leaves 32 squares of one colour and 30 of the other.



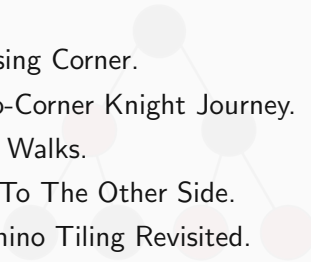
Solution: Deficient Chessboard

- Colour the board as a chessboard.
- Opposite corners have the same colour.
- Removing them leaves 32 squares of one colour and 30 of the other.
- Every domino covers exactly one of each colour.

Solution: Deficient Chessboard

- Colour the board as a chessboard.
- Opposite corners have the same colour.
- Removing them leaves 32 squares of one colour and 30 of the other.
- Every domino covers exactly one of each colour.
- A tiling would need equal colour counts, so it is impossible.

Student Exercises: Colouring

- 
- Basic: One Missing Corner.
 - Easy: Corner-To-Corner Knight Journey.
 - Medium: Board Walks.
 - Hard: Jumping To The Other Side.
 - Very Hard: Domino Tiling Revisited.

Induction and Monovariants



What is Induction?



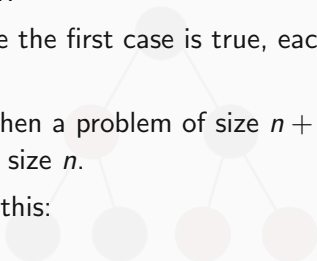
What is Induction?

Induction is a method for proving that a statement is true for every positive integer.

The idea is that once the first case is true, each case forces the next one to be true.

Induction is useful when a problem of size $n + 1$ can be related to the same problem of size n .

It typically goes like this:



What is Induction?

Induction is a method for proving that a statement is true for every positive integer.

The idea is that once the first case is true, each case forces the next one to be true.

Induction is useful when a problem of size $n + 1$ can be related to the same problem of size n .

It typically goes like this:

- **Base case:** prove the statement is true for the smallest value, usually $n = 1$.

What is Induction?

Induction is a method for proving that a statement is true for every positive integer.

The idea is that once the first case is true, each case forces the next one to be true.

Induction is useful when a problem of size $n + 1$ can be related to the same problem of size n .

It typically goes like this:

- **Base case:** prove the statement is true for the smallest value, usually $n = 1$.
- **Induction step:** assume the statement is true for some n , and use this to prove it is true for $n + 1$.

Monovariants

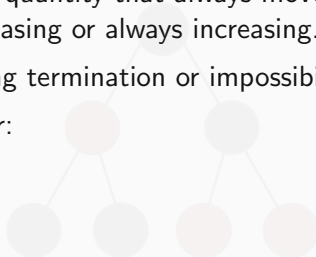


Monovariants

A **monovariant** is a quantity that always moves in one direction, such as always decreasing or always increasing.

It is useful for proving termination or impossibility.

What to look out for:



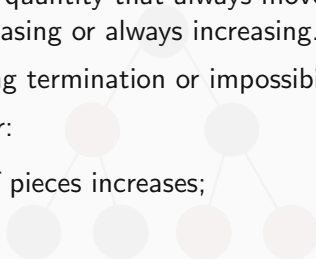
Monovariants

A **monovariant** is a quantity that always moves in one direction, such as always decreasing or always increasing.

It is useful for proving termination or impossibility.

What to look out for:

→ total number of pieces increases;



Monovariants

A **monovariant** is a quantity that always moves in one direction, such as always decreasing or always increasing.

It is useful for proving termination or impossibility.

What to look out for:

- total number of pieces increases;
- distance to a target decreases;

Monovariants

A **monovariant** is a quantity that always moves in one direction, such as always decreasing or always increasing.

It is useful for proving termination or impossibility.

What to look out for:

- total number of pieces increases;
- distance to a target decreases;
- perimeter never increases;

Monovariants

A **monovariant** is a quantity that always moves in one direction, such as always decreasing or always increasing.

It is useful for proving termination or impossibility.

What to look out for:

- total number of pieces increases;
- distance to a target decreases;
- perimeter never increases;
- number of bad pairs decreases.

Worked Example: Chocolate Bar

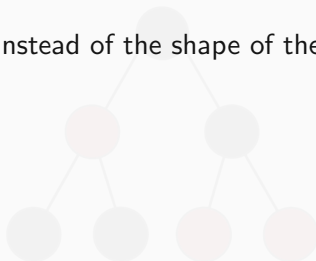
An $n \times m$ chocolate bar must be broken into unit squares.
Each move breaks exactly one existing piece in a straight line.
What is the minimum number of breaks?

Solution: Chocolate Bar



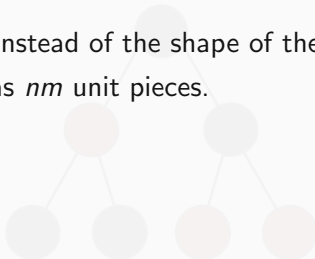
Solution: Chocolate Bar

→ Track progress instead of the shape of the pieces.



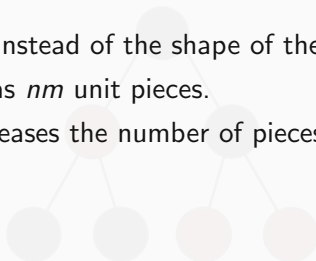
Solution: Chocolate Bar

- Track progress instead of the shape of the pieces.
- The final bar has nm unit pieces.



Solution: Chocolate Bar

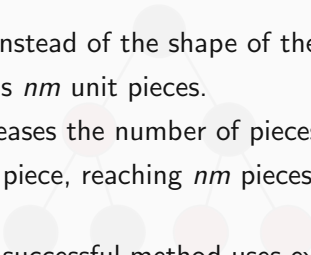
- Track progress instead of the shape of the pieces.
- The final bar has nm unit pieces.
- Each break increases the number of pieces by exactly 1.



Solution: Chocolate Bar

- Track progress instead of the shape of the pieces.
- The final bar has nm unit pieces.
- Each break increases the number of pieces by exactly 1.
- Starting from 1 piece, reaching nm pieces needs $nm - 1$ increases.

Solution: Chocolate Bar

- 
- Track progress instead of the shape of the pieces.
 - The final bar has nm unit pieces.
 - Each break increases the number of pieces by exactly 1.
 - Starting from 1 piece, reaching nm pieces needs $nm - 1$ increases.
 - Therefore every successful method uses exactly $nm - 1$ breaks.

Student Exercises: Induction and Monovariants

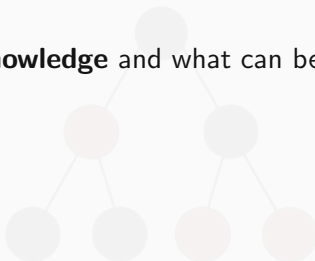
- Basic: Breaking A Chocolate Bar.
- Easy: Induction Inequality $n! > 2^n$.
- Medium: Infected Chessboard.
- Hard: Parliament Pacification.
- Very Hard: Pebble Spreading.

Wrap-Up



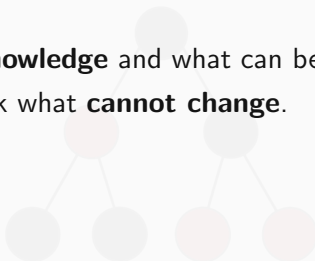
Wrap-Up

→ **Logic** tracks **knowledge** and what can be inferred.



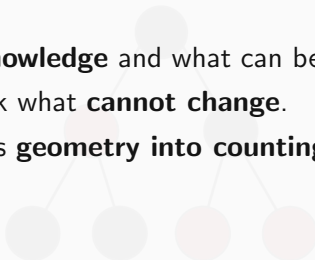
Wrap-Up

- **Logic** tracks **knowledge** and what can be inferred.
- **Invariants** track what **cannot change**.

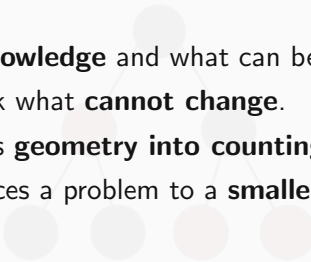


Wrap-Up

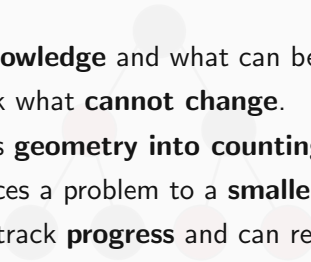
- **Logic** tracks **knowledge** and what can be inferred.
- **Invariants** track what **cannot change**.
- **Colouring** turns **geometry into counting**.



Wrap-Up

- 
- **Logic** tracks **knowledge** and what can be inferred.
 - **Invariants** track what **cannot change**.
 - **Colouring** turns **geometry into counting**.
 - **Induction** reduces a problem to a **smaller case**.

Wrap-Up

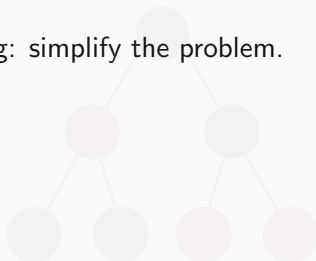
- 
- **Logic** tracks **knowledge** and what can be inferred.
 - **Invariants** track what **cannot change**.
 - **Colouring** turns **geometry into counting**.
 - **Induction** reduces a problem to a **smaller case**.
 - **Monovariants** track **progress** and can reveal **impossibility**.

What's more?



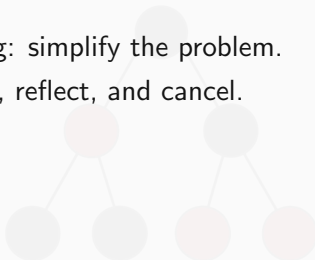
What's more?

→ Wishful thinking: simplify the problem.



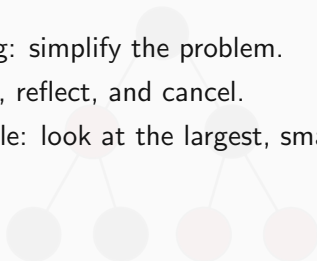
What's more?

- Wishful thinking: simplify the problem.
- Symmetry: pair, reflect, and cancel.

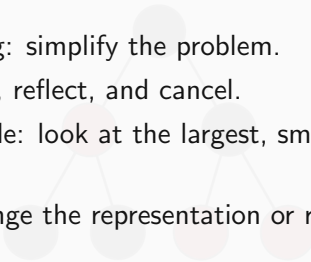


What's more?

- Wishful thinking: simplify the problem.
- Symmetry: pair, reflect, and cancel.
- Extreme principle: look at the largest, smallest, or boundary case.



What's more?

- 
- Wishful thinking: simplify the problem.
 - Symmetry: pair, reflect, and cancel.
 - Extreme principle: look at the largest, smallest, or boundary case.
 - Reduction: change the representation or reduce to a familiar problem.

Thanks for coming!

See ya next week!

I hope you enjoyed the workshop and found some
useful new ways to approach problems.